编译原理与技术课程设计设计概要报告

1. 项目目标与范围

本项目实现 Pascal-S 到 C 的编译程序。系统输入为 Pascal-S 源文件，输出为同名 C 文件，并保证输出文件位于源文件同目录。

本组实现范围覆盖完整编译链路：

1. 词法分析。
2. 语法分析。
3. 语义分析。
4. 语法制导翻译。
5. 代码生成。
6. 错误检测与错误恢复。
7. 单元测试、集成测试与平台评测对齐。

系统交付目标是同时满足开发要求和最终验收要求，具备可运行程序、可复现实验结果和可检查的设计文档。

2. 总体设计原则

本组总体设计遵循“正确性优先、分层解耦、可验证实现、平台约束内落地”的统一原则：首先保证翻译语义正确，确保合法 Pascal-S 程序能够稳定转换为可编译、可运行且功能等价的 C 程序；其次采用前端到后端的分层架构，明确词法、语法、语义、翻译与生成的边界，通过稳定接口降低耦合；同时在设计阶段同步定义模块输入输出、失败路径和测试点，使功能正确性可通过单元测试、集成测试与回归测试持续验证；最后严格对齐课程平台约束，包括可执行文件命名、命令行参数、输出路径和评测流程，确保设计方案从一开始就服务于最终验收通过。

3. 系统总体架构

3.1 处理流程

处理主流程定义如下：

1. 读取命令行参数并定位输入文件。

2. 执行词法分析，输出 Token 流。
3. 执行语法分析，构建语法结构。
4. 执行语义检查，维护符号表并标注语义信息。
5. 执行语法制导翻译与代码生成，输出 C 源码。
6. 写入同名 C 文件。
7. 输出编译过程中的错误与诊断信息。

3.2 模块划分

系统分为六个核心模块：

1. 命令行与文件管理模块。
2. 词法分析模块。
3. 语法分析模块。
4. 语义分析与符号表模块。
5. 语法制导翻译与代码生成模块。
6. 测试与质量验证模块。

4. 模块设计概要

4.1 命令行与文件管理模块

模块职责：

1. 接收并解析运行命令，固定识别 -i 参数，并读取其后给出的 Pascal-S 源文件路径。
2. 检查输入文件是否存在、是否可读取、后缀是否为 .pas；任一条件不满足时，立即给出明确错误信息。
3. 按“输入同名、后缀改为 .c、输出到原目录”的规则生成目标文件路径。
4. 统一输出执行结果（成功或失败）以及失败原因，便于本地调试和平台评测定位问题。

关键约束：

1. 可执行程序文件名必须是 pascc。
2. 评测时该可执行文件必须放在 /data/workspace/myshixun/bin 目录下。

4.2 词法分析模块

模块职责：

1. 按 Pascal-S 词法规则识别关键字、标识符、常量、运算符与界符。

2. 跳过空白符和注释，维护源代码位置信息。
3. 生成可供语法分析消费的 Token 序列。
4. 输出词法错误，至少包含行号、列号与错误类型。

输出定义：

1. Token 类型。
2. Token 属性值。
3. Token 位置信息。

4.3 语法分析模块

模块职责：

1. 使用 YACC / Bison 对 Token 流进行语法归约。
2. 按 Pascal-S 文法构建语法结构。
3. 在语法错误场景下提供可定位诊断。
4. 为语义分析阶段保留可追踪的结构信息。

恢复策略：

1. 对局部错误采用同步恢复，尽量继续分析后续语句。
2. 保证一次编译过程可报告多处错误，不在首错直接终止。

4.4 语义分析与符号表模块

模块职责：

1. 构建并维护多层作用域符号表。
2. 完成重定义、未定义、类型不匹配、参数不匹配等检查。
3. 处理过程/函数声明与调用关系。
4. 将语义信息注入语法结构，供后续生成阶段使用。

核心数据：

1. 符号项：名称、种类、类型、作用域层级、附加属性。
2. 符号表栈：进入/退出作用域时动态维护。

4.5 语法制导翻译与代码生成模块

模块职责：

1. 按语法制导规则将语法语义结构映射为 C 代码。
2. 保持语句级与表达式级语义一致。
3. 输出结构化、可编译的目标 C 程序。

关键映射约定：

1. Pascal `Real` 类型统一映射为 C `float`。
2. 浮点行为按课程要求以“功能翻译正确”为判定标准。
3. 生成文件名与输入同名，后缀变更为 `.c`。

4.6 错误处理机制

统一错误模型覆盖三个阶段：

1. 词法错误：非法字符、非法词素。
2. 语法错误：文法不匹配、结构缺失。
3. 语义错误：符号与类型相关错误。

输出规范：

1. 错误级别。
2. 错误代码或类别。
3. 行号、列号。
4. 简明错误说明与上下文提示。

5. 接口与数据约定

5.1 模块接口约定

接口按“输入明确、输出稳定、异常可追踪”定义：

1. 词法到语法：Token 流接口。
2. 语法到语义：语法结构接口。
3. 语义到生成：带语义标注的中间结构接口。

5.2 数据一致性约定

1. 类型系统在语义分析和代码生成阶段保持统一定义。
2. 标识符解析结果在后续阶段不可被二义重写。
3. 错误定位坐标在各模块中统一为“行号+列号”。

6. 实施与集成方案

6.1 迭代顺序

本组按时间顺序推进，总时长约 60 天：

1. 第 1-10 天：完成需求复核、文法边界确认、目录结构与构建脚本初始化，形成可运行的工程骨架。
2. 第 11-20 天：完成词法分析模块开发与词法测试用例，打通 Token 输出链路。
3. 第 21-32 天：完成语法分析模块开发，接入 YACC / Bison，建立基础错误恢复机制并打通主流程。
4. 第 33-44 天：完成语义分析与符号表实现，补齐类型检查、作用域检查、参数检查等核心语义规则。
5. 第 45-52 天：完成语法制导翻译与代码生成，确保输出 C 文件可编译并满足同名同目录约束。
6. 第 53-60 天：完成联调、回归测试与平台对齐验证，整理交付文档并进行预验收演练。

6.2 组内分工

本组四人分工如下：

1. 金介然：词法分析模块与词法测试用例。
2. 何俊辉：语法分析与语义分析手工编码实现。
3. 李璇：YACC / Bison 语法制导翻译实现。
4. 丁乐航：代码生成、集成测试与进度统筹（组长）。

每名成员对对应模块承担设计、编码、单测与问题修复责任；组长负责接口收口、联调排期与最终交付一致性。